PRACTICAL 2

Aim: Create a Dataset from an Excel file. Apply conditional filtering to extract records where a numerical variable exceeds a given threshold and sort the data descending order.

```
install.packages("readxl")
install.packages("dplyr")

library(readxl)
library(dplyr)

#Importing Excel File
data <- read_excel("C:\\Users\\Student\\Downloads\\BMW sales data (2010-2024).xlsx")

#Structure of the dataset
str(data)
```

```
> str(data)
tibble [50,000 × 11] (S3: tbl_df/tbl/data.frame)
 $ Model               : chr [1:50000] "5 Series" "i8" "5 Series" "X3" ...
 $ Year                : num [1:50000] 2016 2013 2022 2024 2020 ...
 $ Region              : chr [1:50000] "Asia" "North America" "North America" "Middle East" ...
 $ Color               : chr [1:50000] "Red" "Red" "Blue" "Blue" ...
 $ Fuel_Type           : chr [1:50000] "Petrol" "Hybrid" "Petrol" "Petrol" ...
 $ Transmission        : chr [1:50000] "Manual" "Automatic" "Automatic" "Automatic" ...
 $ Engine_Size_L       : num [1:50000] 3.5 1.6 4.5 1.7 2.1 1.9 1.8 1.6 1.7 3 ...
 $ Mileage_KM          : num [1:50000] 151748 121671 10991 27255 122131 ...
 $ Price_USD           : num [1:50000] 98740 79219 113265 60971 49898 ...
 $ Sales_Volume        : num [1:50000] 8300 3428 6994 4047 3080 ...
 $ Sales_Classification: chr [1:50000] "High" "Low" "Low" "Low" ...
```

```
#Converting to DataFrame
df<-as.data.frame(data)
View(df)

#Applying conditional filtering (Price_USD > 50,000)
filtered_data <- df %>%
  filter(Price_USD > 50000)

#Sort Filtered data in descending order
sorted_data <- filtered_data %>%
  arrange(desc(Price_USD))

#View the result
head(sorted_data)
```

```
> head(sorted_data)
     Model Year        Region  Color Fuel_Type Transmission Engine_Size_L Mileage_KM Price_USD
1       i8 2010   Middle East Silver  Electric       Manual           4.2     115320    119998
2       i8 2024        Africa Silver    Diesel    Automatic           4.1     163849    119997
3       X6 2019          Asia    Red  Electric    Automatic           3.3     142419    119997
4       X1 2016        Africa   Grey    Petrol       Manual           4.1     172950    119996
5 3 Series 2019   Middle East  White    Hybrid       Manual           4.0      12264    119994
6       i8 2016 North America    Red    Hybrid       Manual           4.6      26622    119992
  Sales_Volume Sales_Classification
1         5842                  Low
2         9250                 High
3         4575                  Low
4         9620                 High
5         3259                  Low
6         8065                 High
```

Roll No. 35

PRACTICAL 3

Aim: Create a Dataset from an Excel file. Apply conditional filtering to extract records where a numerical variable exceeds a given threshold and sort the data descending order.

```
# Creating employee salary dataset
emp <- data.frame(
  EmployeeID  = c(101, 102, 103, 104, 105),
  Name        = c("Aaditya", "Parth", "Padmaj", "Anushka", "Amrita"),
  Department  = c("Data Scientist",
                  "Computer Architecture",
                  "Computer Networks",
                  "Data Architect",
                  "UI Designer"),
  BasicSalary = c(60000, 55000, 55000, 65000, 50000),
  HRA         = c(5500, 4500, 3500, 2200, 4200),
  Bonus       = c(7000, 5000, 3000, 4500, 3500),
  Age         = c(19, 19, 18, 19, 19)
)

# View Dataset
emp
```

	EmployeeID	Name	Department	BasicSalary	HRA	Bonus	Age
1	101	Aaditya	Data Scientist	60000	5500	7000	19
2	102	Parth	Computer Arcitechture	55000	4500	5000	19
3	103	Padmaj	Computer Networks	55000	3500	3000	18
4	104	Anushka	Data Architect	65000	2200	4500	19
5	105	Amrita	UI Designer	50000	4200	3500	19

```
# Select salary-related columns
emp_selected <- emp[, c("EmployeeID", "Department",
                        "BasicSalary", "HRA", "Bonus")]

# View selected data
emp_selected
```

	EmployeeID	Department	BasicSalary	HRA	Bonus
1	101	Data Scientist	60000	5500	7000
2	102	Computer Arcitechture	55000	4500	5000
3	103	Computer Networks	55000	3500	3000
4	104	Data Architect	65000	2200	4500
5	105	UI Designer	50000	4200	3500

Roll No. 35

```
# Calculate Total Salary
emp_selected$TotalSalary <-
  emp_selected$BasicSalary +
  emp_selected$HRA +
  emp_selected$Bonus
```

```
  EmployeeID            Department BasicSalary  HRA Bonus TotalSalary
1        101        Data Scientist       60000 5500  7000       72500
2        102 Computer Arcitechture       55000 4500  5000       64500
3        103     Computer Networks       55000 3500  3000       61500
4        104        Data Architect       65000 2200  4500       71700
5        105           UI Designer       50000 4200  3500       57700
```

```
# Calculate Annual Salary
emp_selected$AnnualSalary <- emp_selected$TotalSalary * 12
```

```
> emp_selected
  EmployeeID            Department BasicSalary  HRA Bonus TotalSalary AnnualSalary
1        101        Data Scientist       60000 5500  7000       72500       870000
2        102 Computer Arcitechture       55000 4500  5000       64500       774000
3        103     Computer Networks       55000 3500  3000       61500       738000
4        104        Data Architect       65000 2200  4500       71700       860400
5        105           UI Designer       50000 4200  3500       57700       692400
>
```

```
# View structure
str(emp_selected)

# View final dataset
emp_selected
```

```
> str(emp_selected)
'data.frame':   5 obs. of  7 variables:
 $ EmployeeID  : num  101 102 103 104 105
 $ Department  : chr  "Data Scientist" "Computer Arcitechture" "Computer Networks" "Data Architect" ...
 $ BasicSalary : num  60000 55000 55000 65000 50000
 $ HRA         : num  5500 4500 3500 2200 4200
 $ Bonus       : num  7000 5000 3000 4500 3500
 $ TotalSalary : num  72500 64500 61500 71700 57700
 $ AnnualSalary: num  870000 774000 738000 860400 692400
```

PRACTICAL 4

Aim: Identify and remove duplicates observation from a dataset. Display the numbers of records before and removing duplicates.

```
#Import Dataset
data<-read.csv("C:\\Users\\Student\\Downloads\\Iris.csv")
View(data)

# View structure
str(data)
```

```
> str(data)
'data.frame':   150 obs. of  6 variables:
 $ Id           : int  1 1 3 1 5 1 7 1 9 10 ...
 $ SepalLengthCm: num  5.1 5.1 4.7 5.1 5 5.1 NA 5.1 4.4 4.9 ...
 $ SepalWidthCm : num  3.5 3.5 NA 3.5 3.6 3.5 3.4 3.5 2.9 3.1 ...
 $ PetalLengthCm: num  1.4 1.4 1.3 1.4 1.4 1.4 1.4 1.4 1.4 1.5 ...
 $ PetalWidthCm : num  0.2 0.2 0.2 0.2 0.2 0.2 0.3 0.2 0.2 0.1 ...
 $ Species      : chr  "Iris-setosa" "Iris-setosa" "Iris-setosa" "Iris-setosa" ...
```

```
#Count Records Before Removing Duplicates
# Number of records before removing duplicates
before_count <- nrow(data)
before_count
```

```
> before_count
[1] 150
```

```
# Identify Duplicate Observations
# Identify duplicate rows
duplicate_rows <- duplicated(data)

# Count duplicate rows
sum(duplicate_rows)
```

```
> # Count duplicate rows
> sum(duplicate_rows)
[1] 5
```

```
# Remove Duplicate Observations
# Remove duplicate rows
data_unique <- data[!duplicated(data), ]

# Number of records after removing duplicates
after_count <- nrow(data_unique)
after_count
```

```
> after_count
[1] 145
```

Roll No. 35

```
# Display record count comparison
record_summary <- data.frame(
  Before_Removing_Duplicates = before_count,
  After_Removing_Duplicates = after_count
)

record_summary
```

```
> record_summary
  Before_Removing_Duplicates After_Removing_Duplicates
1                        150                       145
>
```

PRACTICAL 5

Aim: Reshape a wide-format dataset into long format and then convert it back to wide format using appropriate R functions.

```
install.packages("tidyr")
library(tidyr)

student_marks<-data.frame(
  Student_ID=c(101,102,103),
  Math=c(78,85,90),
  Science=c(82,88,91),
  English=c(75,80,89)
)

student_marks
```

```
> student_marks
  Student_ID Math Science English
1        101   78      82      75
2        102   85      88      80
3        103   90      91      89
```

```
long_data<-pivot_longer(
  student_marks,
  cols=c(Math, Science, English),
  names_to="Subject",
  values_to="Marks"
)

long_data
```

```
> long_data
# A tibble: 9 × 3
  Student_ID Subject Marks
       <dbl> <chr>   <dbl>
1        101 Math       78
2        101 Science    82
3        101 English    75
4        102 Math       85
5        102 Science    88
6        102 English    80
7        103 Math       90
8        103 Science    91
9        103 English    89
```

```
wide_data <- pivot_wider(
  long_data,
  names_from = Subject,
```

```
  values_from = Marks
)

wide_data
```

```
> wide_data
# A tibble: 3 × 4
  Student_ID  Math Science English
       <dbl> <dbl>   <dbl>   <dbl>
1        101    78      82      75
2        102    85      88      80
3        103    90      91      89
```

PRACTICAL 6

Aim: Generate descriptive statistics and frequency tables for categorical and numerical variables. Interpret the result briefly in comments.

```
data <- data.frame(
  employee_id = 1:10,
  age = c(20,50,20,25,30,26,40,50,60,70),
  salary = c(20000,30000,50000,80000,40000,40000,70000,60000,50000,30000),
  department = c("IT","IT","Java","Python","Computer Network","HR","Manager","UI","Developer","Operator"),
  gender = c("Male","Female","Male","Female","Male","Female","Male","Female","Male","Female"))

# View data
data
str(data)
```

```
> data
   employee_id age salary       department gender
1            1  20  20000               IT   Male
2            2  50  30000               IT Female
3            3  20  50000             Java   Male
4            4  25  80000           Python Female
5            5  30  40000 Computer Network   Male
6            6  26  40000               HR Female
7            7  40  70000          Manager   Male
8            8  50  60000               UI Female
9            9  60  50000        Developer   Male
10          10  70  30000         Operator Female
> str(data)
'data.frame':   10 obs. of  5 variables:
 $ employee_id: int  1 2 3 4 5 6 7 8 9 10
 $ age        : num  20 50 20 25 30 26 40 50 60 70
 $ salary     : num  20000 30000 50000 80000 40000 40000 70000 60000 50000 30000
 $ department : chr  "IT" "IT" "Java" "Python" ...
 $ gender     : chr  "Male" "Female" "Male" "Female" ...
```

```
# Summary statistics
summary(data$age)
summary(data$salary)

mean(data$age)
median(data$age)
sd(data$age)

mean(data$salary)
median(data$salary)
sd(data$salary)
```

```
> summary(data$age)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  20.00   25.25   35.00   39.10   50.00   70.00
> summary(data$salary)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  20000   32500   45000   47000   57500   80000
> mean(data$age)
[1] 39.1
> median(data$age)
[1] 35
> sd(data$age)
[1] 17.67893
> mean(data$salary)
[1] 47000
> median(data$salary)
[1] 45000
> sd(data$salary)
[1] 18885.62
```

Roll No. 35

# Frequency tables
table(data$department)
table(data$gender)

```
> table(data$department)

Computer Network        Developer               HR               IT             Java          Manager         Operator           Python
               1                1                1                2                1                1                1                1
              UI
               1
> table(data$gender)

Female   Male
     5      5
```

Percentage tables
prop.table(table(data$department)) * 100
prop.table(table(data$gender)) * 100

```
> prop.table(table(data$department)) * 100

Computer Network        Developer               HR               IT             Java          Manager         Operator           Python
              10               10               10               20               10               10               10               10
              UI
              10
> prop.table(table(data$gender)) * 100

Female   Male
    50     50
```

Psych package
install.packages("psych") # run once
library(psych)

describe(data[, c("age", "salary")])

```
> describe(data[, c("age", "salary")])
       vars  n     mean       sd median  trimmed      mad   min   max range skew kurtosis      se
age       1 10     39.1    17.68     35    37.62    22.24    20    70    50 0.39    -1.49    5.59
salary    2 10 47000.0 18885.62  45000 46250.00 22239.00 20000 80000 60000 0.30    -1.27 5972.16
```

Roll No. 35

PRACTICAL 7

Aim: Perform a one-sample t-test and an independent two-sample t-test on a suitable dataset. State Null and alternative hypotheses and interpret the result.

Practical 7

```
#Custom dataset
#Performing this test for the Single sample.
scores <- c(65,70,72,68,75,78,74,69,71,73)
t.test(scores, mu = 70)
```

```
        One Sample t-test

data:  scores
t = 1.2652, df = 9, p-value = 0.2376
alternative hypothesis: true mean is not equal to 70
95 percent confidence interval:
 68.81807 74.18193
sample estimates:
mean of x
     71.5
```

```
#For Double dataset
group_A <- c(70,72,68,75,74,71,69)
group_B <- c(65,67,70,66,68,64,69)
t.test(group_A,group_B,var.equal = TRUE)
```

```
        Two Sample t-test

data:  group_A and group_B
t = 3.3824, df = 12, p-value = 0.005444
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 1.525027 7.046402
sample estimates:
mean of x mean of y
 71.28571  67.00000
```

Roll No. 35

Practical 7' (Optional)

```
#For Random: Single Dataset
set.seed(123)

scores <- rnorm(1000, mean= 72, sd = 10)

t_test_one <- t.test(scores, mu = 70)
t_test_one
```

```
        One Sample t-test

data:  scores
t = 6.8918, df = 999, p-value = 9.744e-12
alternative hypothesis: true mean is not equal to 70
95 percent confidence interval:
 71.54589 72.77667
sample estimates:
mean of x
 72.16128
```

```
#For Random: Double Dataset
set.seed(456)

male_scores <- rnorm(1000,mean = 68, sd = 9)
female_scores<-rnorm(1000, mean=72, sd=8)

t_test_two<-t.test(male_scores, female_scores)

t_test_two
```

```
        Welch Two Sample t-test

data:  male_scores and female_scores
t = -10.586, df = 1980.3, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -4.734623 -3.254511
sample estimates:
mean of x mean of y
 68.50840  72.50297
```

PRACTICAL 8

Aim: Conduct a Chi-Square test of independence using a contingency table. Display expected frequencies and interpret the test result.

```
#Contigency Table
job_table<-matrix(
  c(220, 180, 200,
    240, 260, 300),
  nrow=2,
  byrow=TRUE
)

#Adding Row and Column Names
rownames(job_table)<-c("Male", "Female")
colnames(job_table)<-c("IT", "Management", "Government")

job_table

chi_test<-chisq.test(job_table)
chi_test
chi_test$expected
```

```
> job_table<-matrix(
+   c(220, 180, 200,
+     240, 260, 300),
+   nrow=2,
+   byrow=TRUE
+ )
> rownames(job_table)<-c("Male", "Female")
> colnames(job_table)<-c("IT", "Management", "Government")
>
> job_table
        IT Management Government
Male   220        180        200
Female 240        260        300
>
> chi_test<-chisq.test(job_table)
> chi_test

        Pearson's Chi-squared test

data:  job_table
X-squared = 6.9862, df = 2, p-value = 0.03041

> chi_test$expected
             IT Management Government
Male   197.1429   188.5714   214.2857
Female 262.8571   251.4286   285.7143
```

Roll No. 35

PRACTICAL 9

Aim: Create a visualization including histogram, box plot, and scatter plot using ggplot2. Customize labels and titles appropriately.

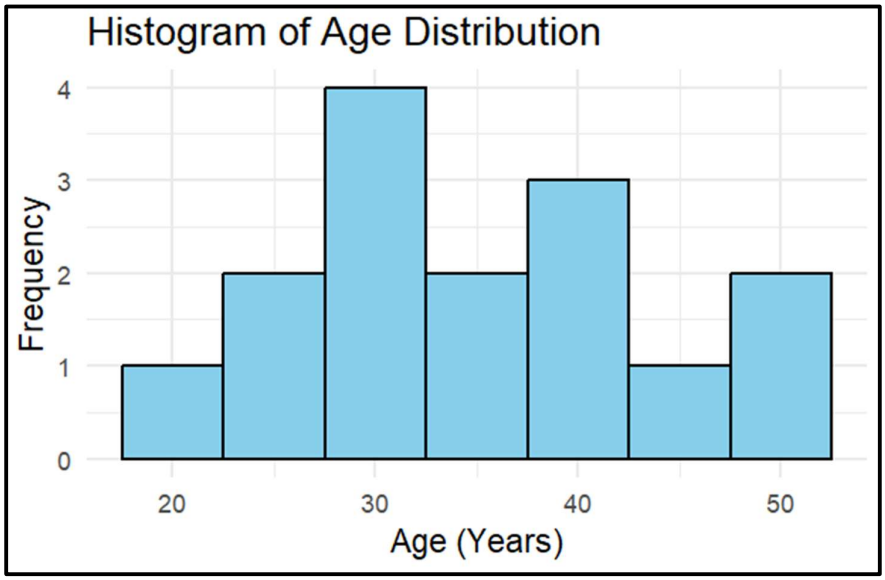
```
install.packages("ggplot2")
library(ggplot2)

Create sample dataset
data <- data.frame(
  Age = c(22,25,30,35,28,40,45,50,32,27,29,34,38,42,48),
  Salary =
c(25000,28000,35000,40000,30000,45000,50000,55000,37000,29000,31000,38000,42000,47000,52000)
)
View(data)
```

	Age	Salary
1	22	25000
2	25	28000
3	30	35000
4	35	40000
5	28	30000
6	40	45000
7	45	50000
8	50	55000
9	32	37000
10	27	29000
11	29	31000
12	34	38000
13	38	42000
14	42	47000
15	48	52000

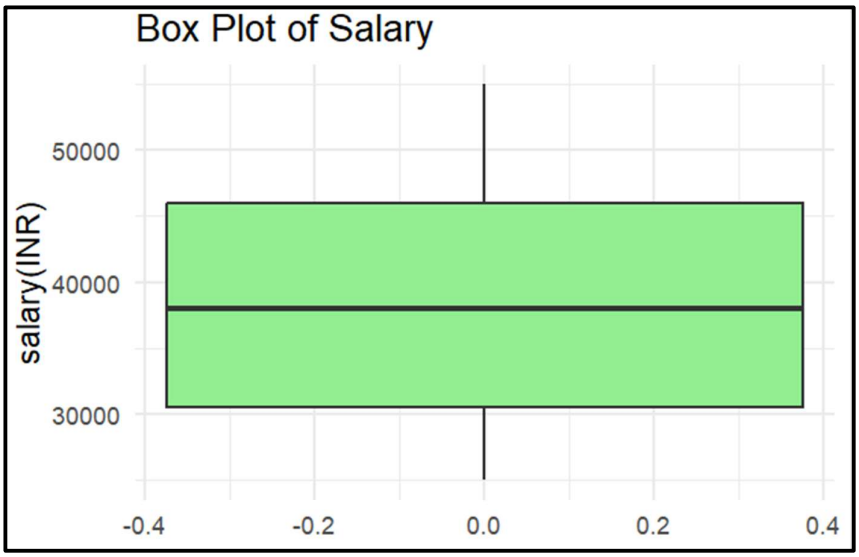
1.Histogram(Distribution of Age)

```
ggplot(data, aes(x=Age)) +
  geom_histogram(binwidth = 5,fill ="skyblue",color = "black")+
  labs(
    title="Histogram of Age Distribution",
    x = "Age (Years)",
    y = "Frequency"
  )+
  theme_minimal()
```



2.Box Plot (Salary Distribution)

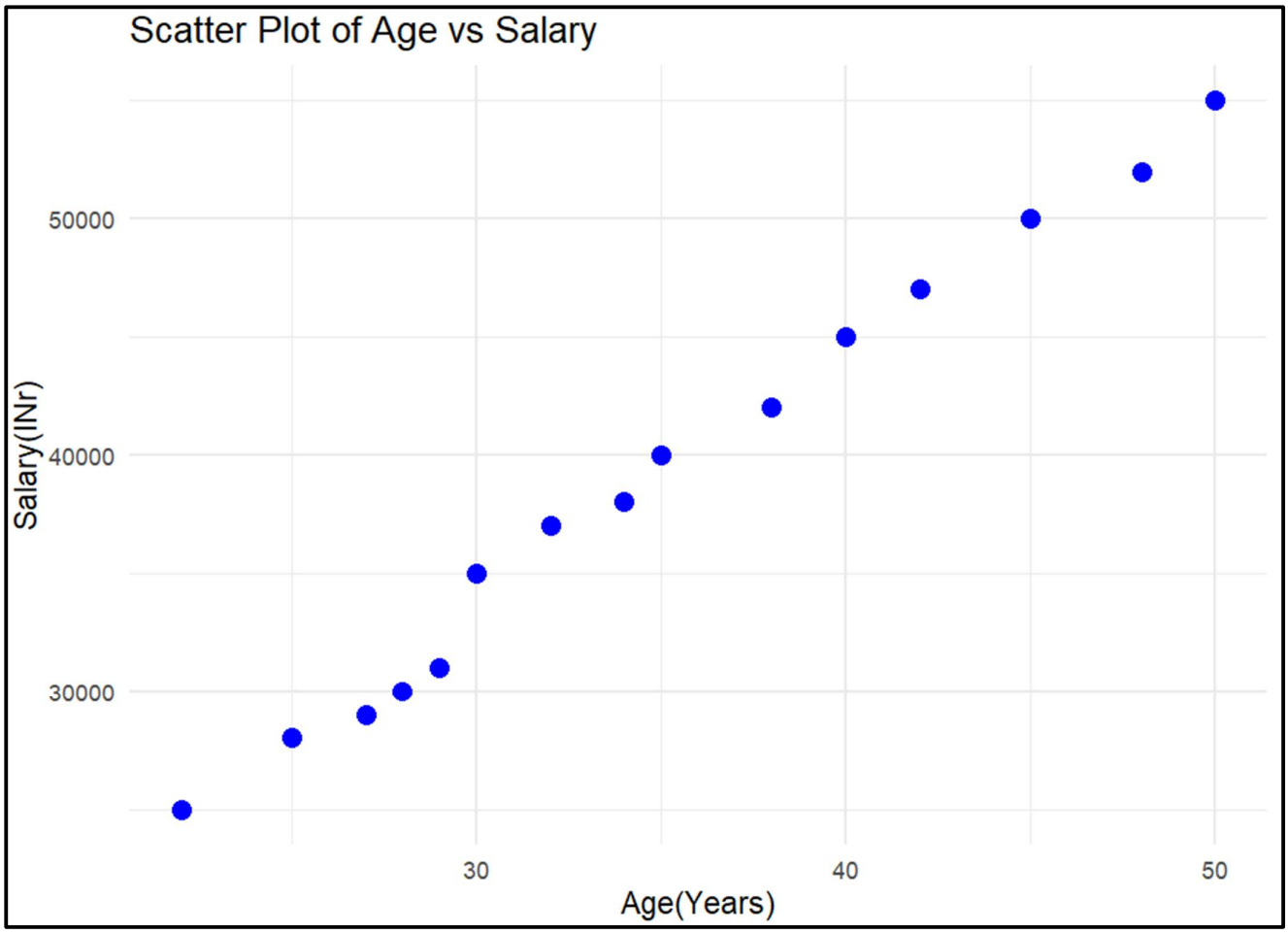
```
ggplot(data, aes(y=Salary))+
  geom_boxplot(fill = "lightgreen")+
  labs(
    title="Box Plot of Salary",
    y="salary(INR)"
  )+
  theme_minimal()
```



Roll No. 35

3.Scatter Plot(Age vs Salary)

```
ggplot(data, aes(x=Age, y=Salary))+
  geom_point(color="blue", size=3)+
  labs(
    title="Scatter Plot of Age vs Salary",
    x="Age(Years)",
    y="Salary(INr)"
  )+
  theme_minimal()
```



Roll No. 35

PRACTICAL 10

Aim: Perform linear regression and logistic regression analysis on a dataset. Interpret coefficient and export the result to a CSV file

```
#Create a sample dataset
data <- data.frame(
  Hours_Studied = c(2,4,6,8,10,12,14,16,18,20),
  Marks = c(35,40,50,55,65,70,75,85,88,95),
  Passed = c(0,0,0,1,1,1,1,1,1,1)
)

#View dataset
data
```

```
> data
   Hours_Studied Marks Passed
1              2    35      0
2              4    40      0
3              6    50      0
4              8    55      1
5             10    65      1
6             12    70      1
7             14    75      1
8             16    85      1
9             18    88      1
10            20    95      1
>
```

```
#Linear Regression
linear_model <- lm(Marks ~ Hours_Studied, data = data)
#Model summary
summary(linear_model)
#Convert coefficients to dataframe
linear_results <- as.data.frame(summary(linear_model)$coefficients)
```

```
> #Linear Regression
> linear_model <- lm(Marks ~ Hours_Studied, data = data)
> #Model summary
> summary(linear_model)

Call:
lm(formula = Marks ~ Hours_Studied, data = data)

Residuals:
    Min      1Q  Median      3Q     Max 
-2.1273 -1.1636 -0.5091  1.0364  2.5818 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)   28.60000    1.17060   24.43 8.41e-09 ***
Hours_Studied  3.38182    0.09433   35.85 4.01e-10 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 1.714 on 8 degrees of freedom
Multiple R-squared:  0.9938,	Adjusted R-squared:  0.993 
F-statistic:  1285 on 1 and 8 DF,  p-value: 4.013e-10
```

Roll No. 35

#Export to csv
write.csv(linear_results, "linear_regression_results.csv")
linear_results

logistic_model <- glm(Passed ~ Hours_Studied,data=data,family=binomial)

```
> #Convert coefficients to dataframe
> linear_results <- as.data.frame(summary(linear_model)$coefficients)
> #Export to csv
> write.csv(linear_results, "linear_regression_results.csv")
> linear_results
              Estimate Std. Error  t value     Pr(>|t|)
(Intercept)   28.600000 1.17059943 24.43193 8.408903e-09
Hours_Studied  3.381818 0.09432959 35.85109 4.013266e-10
>
> logistic_model <- glm(Passed ~ Hours_Studied,data=data,family=binomial)
Warning messages:
1: glm.fit: algorithm did not converge
2: glm.fit: fitted probabilities numerically 0 or 1 occurred
>
> summary(logistic_model)
```

summary(logistic_model)

odds_ratios <- exp(coef(logistic_model))
odds_ratios

logistic_results <- as.data.frame(summary(logistic_model)$coefficients)
write.csv(logistic_results,"logistic_regression_results.csv")
logistic_results

```
> summary(logistic_model)

Call:
glm(formula = Passed ~ Hours_Studied, family = binomial, data = data)

Coefficients:
               Estimate Std. Error z value Pr(>|z|)
(Intercept)     -154.40  187553.97  -0.001    0.999
Hours_Studied     22.05   26336.11   0.001    0.999

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1.2217e+01  on 9  degrees of freedom
Residual deviance: 1.0636e-09  on 8  degrees of freedom
AIC: 4

Number of Fisher Scoring iterations: 25

>
> odds_ratios <- exp(coef(logistic_model))
> odds_ratios
  (Intercept) Hours_Studied
 8.845513e-68  3.765843e+09
>
> logistic_results <- as.data.frame(summary(logistic_model)$coefficients)
> write.csv(logistic_results,"logistic_regression_results.csv")
> logistic_results
               Estimate Std. Error       z value  Pr(>|z|)
(Intercept)   -154.39588  187553.97 -0.0008232077 0.9993432
Hours_Studied   22.04924   26336.11  0.0008372244 0.9993320
>
```

Roll No. 35